



School of Computer Science and Engineering

SUMMER TRAINING/INTERNSHIP PROJECT REPORT

(Term June-July 2026)

NaviPath – Terminal-Based Shortest Path & City Navigation System

Graph Algorithms and Shortest Path Optimization in C++17

NAME	REGISTRATION NUMBER
Shaurya Sharma	12412213

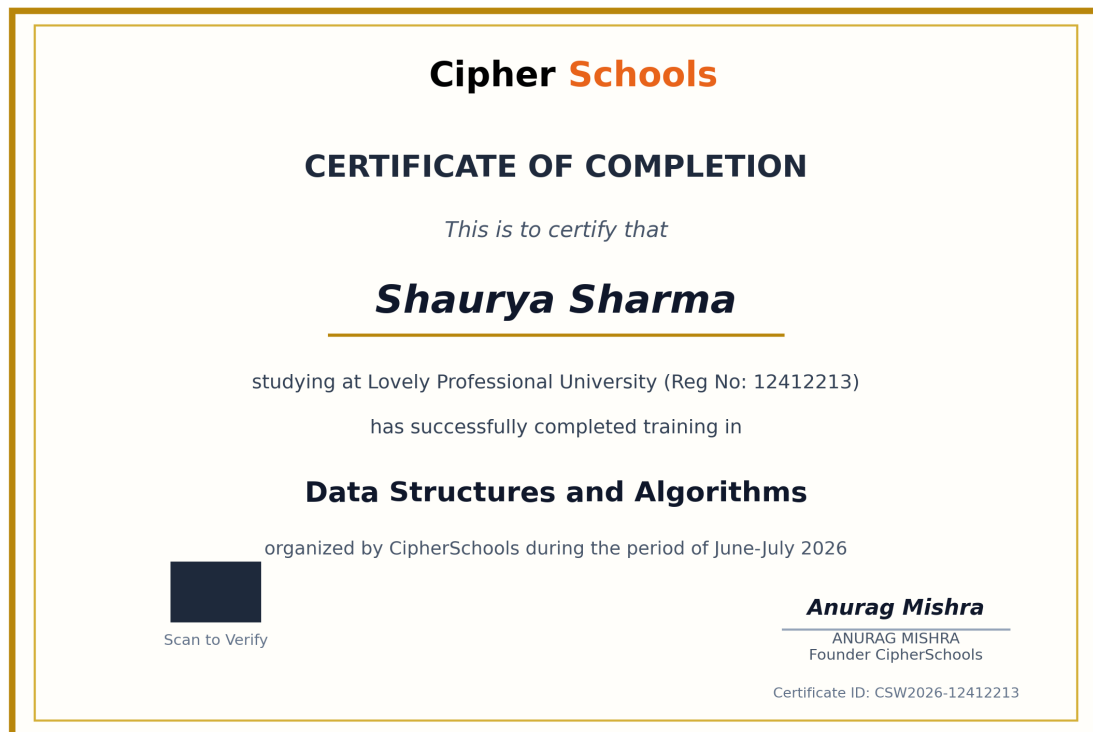
Course Code : CSE433

Under the Guidance of

– CIPHERSCHOOLS

CERTIFICATE OF COMPLETION

Awarded by the Centre for Professional Enhancement, CipherSchools.



This certificate, issued upon successful completion of the Summer Training program, reflects the strong technical foundation that underpins the project presented in this report — from object-oriented system design and algorithm implementation to terminal UI development, graph data structures, and Dijkstra's shortest path algorithm in C++17.

DECLARATION

I, **Shaurya Sharma** (Reg. No. **12412213**), hereby declare that the Summer Training/Internship Project Report entitled "**NaviPath – Terminal-Based Shortest Path & City Navigation System**" submitted to the School of Computer Science and Engineering, Lovely Professional University, is a record of original work carried out by me during the period June–July 2026.

I further declare that this report, or any part thereof, has not been submitted to any other University or Institute for the award of any degree, diploma, or similar title, and that all sources of information used have been duly acknowledged.

Shaurya Sharma (Reg. No. 12412213)

Date: July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and the faculty members of the School of Computer Science and Engineering for their invaluable guidance, constant supervision, and continuous support throughout the duration of our summer internship. Their encouragement and insightful feedback have been instrumental in the successful completion of our project titled "**NaviPath – Terminal-Based Shortest Path & City Navigation System.**"

I am especially thankful for the resources, constructive suggestions, and the structured training in Data Structures and Algorithms provided by **CipherSchools** (13th June – 24th July 2026) that enabled us to strengthen our programming foundation and apply it directly to this project.

I also extend my heartfelt thanks to Lovely Professional University and the Department of Computer Science and Engineering for providing the opportunity to undertake this internship project and for fostering an environment that encourages learning and innovation.

TABLE OF CONTENTS

Declaration	3
Certificate	2
Acknowledgement	4
Chapter 1: Introduction of Organization	6
1.1 About CipherSchools	6
1.2 Overview of Training Domain	6
1.3 Objective of the Project	6
Chapter 2: Summer Training / Internship Content Detail	7
2.1 Tools & Technologies Used	7
2.2 Areas Covered During Training	7
2.3 Summary of Work	7
Chapter 3: Summer Training / Internship Project Detail	8
3.1 Problem Statement	8
3.2 Scope and Objectives	8
3.3 System Architecture and Layer Breakdown	8
3.4 Methodology / Design	9
3.5 Project Outcomes	9
3.6 Technologies Used	9
Chapter 4: Source Code and System Snapshots	10
4.1 Graph & Dijkstra Algorithm Code	10
4.2 NavigationSystem – Core Routing Engine	10
4.3 Model Layer – Key Class Structures	11
4.4 System Architecture & Terminal Snapshots	11
Chapter 5: Bibliography & Project Links	12

CHAPTER 1: INTRODUCTION OF ORGANIZATION

1.1 About CipherSchools

The online summer internship was an intensive training experience focused on Data Structures and Algorithms (DSA), organized by **CipherSchools** from 13th June till 24th July 2026. The program combined structured learning with continuous practical problem-solving, helping us strengthen our programming foundation and analytical thinking.

This project was carried out as part of the Summer Internship Course conducted from June to July 2026. The internship simulated a real-world software engineering scenario, treating city transportation and route optimization as the target industry. The objective was to apply theoretical concepts of Data Structures, Graph Theory, Greedy & Priority-Queue Algorithms, Object-Oriented Programming, and Modular Architecture learned throughout the curriculum to a comprehensive terminal-based navigation system.

1.2 Overview of Training Domain

The online summer internship provided an intensive hands-on experience in Data Structures and Algorithms (DSA). Throughout the internship, we followed a consistent routine of solving approximately five LeetCode problems daily, supported by regular homework and practice exercises. This helped us improve our ability to analyze complex computational problems, identify suitable algorithmic approaches, write efficient code, and debug systematically.

The internship emphasized the practical value of DSA in developing logical thinking and algorithmic efficiency. Working with different data structures (Graphs, Priority Queues, Adjacency Lists, Hash Maps) and optimization techniques strengthened our understanding of how to manage data effectively, optimize graph traversals, and approach computational pathfinding with optimal time and space complexity.

1.3 Objective of the Project

The primary objective was to develop a comprehensive, terminal-based navigation and route optimization system in C++17 demonstrating real-world software engineering principles. This involved:

- Designing a 4-layer modular architecture separating presentation, control, business logic (DSA engine), and data models.
- Implementing an Adjacency List graph representation for city locations, distance weights, and road speed limits.
- Implementing Dijkstra's Shortest Path Algorithm with a Min-Priority Queue for fast $\mathcal{O}((V+E)\log V)$ performance.
- Supporting dual optimization criteria: Shortest Physical Distance (km) and Fastest Travel Time (minutes).
- Providing detailed step-by-step turn-by-turn guidance and visual ASCII map schematics.

CHAPTER 2: SUMMER TRAINING / INTERNSHIP CONTENT DETAIL

2.1 Tools & Technologies Used

- **Programming Language:** C++17 (Standard Template Library, `<<`, `>>`, `<<<`)
- **Build Systems:** CMake & GNU Make
- **Development Environment:** VS Code, GCC / MSVC Compiler
- **Version Control:** Git & GitHub
- **Terminal Interface:** ANSI Escape Codes, Clean CLI Box Drawing

2.2 Areas Covered During Training

- **Data Structures & Algorithms:** Daily LeetCode practice (~5 problems/day across Graph Theory, Dynamic Programming, Trees, Priority Queues).
- **Object-Oriented Programming:** Encapsulation, abstraction, separation of concerns, modular header/source design.
- **Graph Theory & Pathfinding:** Adjacency lists, edge weighting, Dijkstra's algorithm, priority queues, path backtracking.
- **Software Architecture Design:** Clean multi-layer design isolating presentation, application logic, and core algorithms.

2.3 Summary of Work

The project followed a structured, modular development workflow. It commenced with architectural planning — defining the multi-layer design (Presentation, Controller, Core Graph Logic, Data Models) — followed by model design for core navigation entities (Landmark, Edge, Graph, DijkstraResult).

The core algorithmic layer was implemented in `Graph.cpp`, featuring Dijkstra's algorithm accelerated by a Min-Heap Priority Queue (`std::priority_queue`). Predecessor tracking was built to reconstruct the exact path back to the origin.

Key Outcomes:

- Improved consistency in solving complex algorithmic & graph problems.
- Strengthened understanding of graph data structures and priority queues.
- Applied learning in a clean C++17 codebase pushed to GitHub with clean build automation.

CHAPTER 3: SUMMER TRAINING / INTERNSHIP PROJECT DETAIL

3.1 Problem Statement

NaviPath is a terminal-based shortest path and city navigation system developed in C++17. Modern navigation applications require fast, accurate route calculations under different constraints (such as distance minimization vs travel time optimization considering speed limits). Rather than being designed as a simplistic menu-driven program, NaviPath is structured as a modular application with clean graph representation, Dijkstra algorithm optimization, predecessor path tracking, and interactive CLI rendering.

3.2 Scope and Objectives

Scope: Full simulation of a city navigation system including landmark mapping, road adjacency view, turn-by-turn route guidance, ASCII map schematic, and dynamic addition of custom locations and road connections.

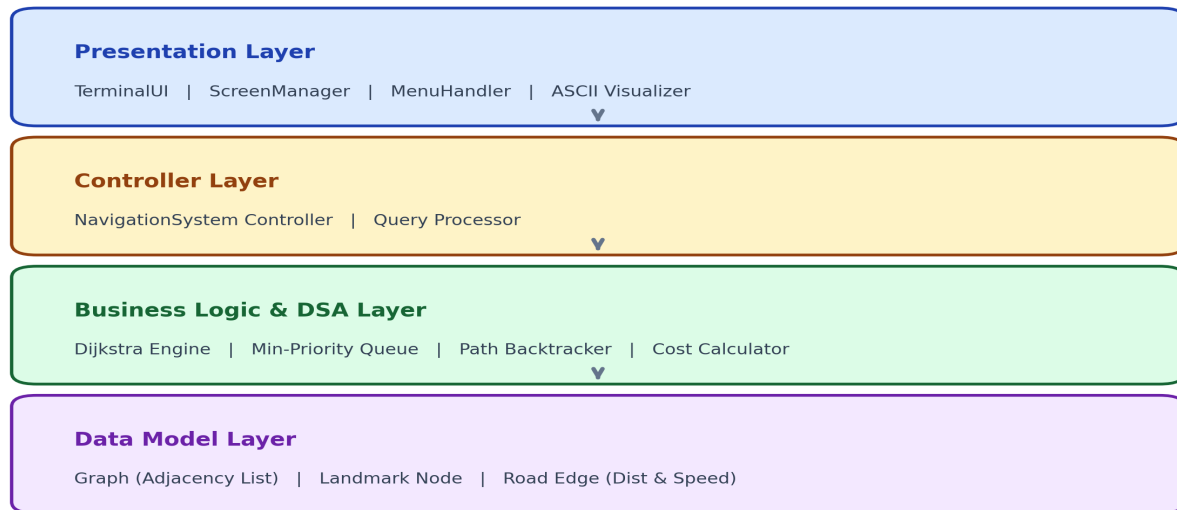
Objectives:

- Design and implement an Adjacency List graph structure with dual edge metrics (distance & speed limit).
- Implement Dijkstra's algorithm with Min-Priority Queue achieving $\mathcal{O}((V+E)\log V)$ time complexity.
- Support dual optimization modes (Shortest Distance in km and Fastest Travel Time in minutes).
- Provide turn-by-turn navigation output detailing individual segment distances, speed limits, and time estimates.

3.3 System Architecture and Layer Breakdown

The overall application architecture follows a 4-layer design:

Layer	Components	Responsibility
Presentation	TerminalUI, MenuHandler, ASCII Map Render	CLI menu rendering, input parsing, turn-by-turn route display
Controller	NavigationSystem Controller	Coordinates user requests, invokes graph queries, formats output
Business Logic / DSA	Graph Engine, Dijkstra Algorithm, Priority Queue	Shortest path traversal, cost relaxation, predecessor tracking
Model	Landmark, Edge, DijkstraResult	Data entities representing vertices, weighted edges, and search results



3.4 Methodology / Design (Personal Contribution)

As the primary developer of this project, my main focus was on designing the **Graph Data Structure** and implementing the **Dijkstra Algorithmic Core**.

Dijkstra's Algorithm Implementation:

The graph is represented using an adjacency list `std::vector<std::vector<std::priority_queue<vector<, greater>>>`. Dijkstra's algorithm uses a min-priority queue `std::priority_queue<vector<, greater>>` to continuously extract the node with the minimum distance or travel time.

During relaxation, if `costs[u] + edgeWeight < costs[v]`, the distance/time array is updated, and the predecessor `predecessors[v] = u` is recorded. Upon reaching the destination, the route is reconstructed by stepping backward from the destination to the source, then reversing the sequence.

3.5 Project Outcomes

Key Technical Outcomes:

- Complete navigation workflow supporting both shortest distance and fastest travel time queries.
- High algorithmic performance: $\mathcal{O}((V+E)\log V)$ query execution time.
- Interactive turn-by-turn directions detailing road names, speed limits, and segment travel times.
- Cross-platform compilation using standard C++17 tools (GCC, Clang, MSVC, CMake).

3.6 Technologies Used

Layer	Technology
Programming Language	C++17
Build Systems	CMake, GNU Make
Core Data Structures	Graph Adjacency List, Min-Priority Queue, Hash Map
Terminal UI	ANSI Escape Codes, Clean Box Drawing
Version Control	Git & GitHub

CHAPTER 4: SOURCE CODE AND SYSTEM SNAPSHOTS

4.1 Graph & Dijkstra Algorithm Code

File: *src/Graph.cpp* (Core Dijkstra Implementation)

```
DijkstraResult Graph::runDijkstra(int startNode, bool optimizeForTime) const {
    DijkstraResult result;
    const double INF = std::numeric_limits::infinity();
    result.costs.assign(numVertices, INF);
    result.distances.assign(numVertices, INF);
    result.travelTimes.assign(numVertices, INF);
    result.predecessors.assign(numVertices, -1);

    using QueueElement = std::pair;
    std::priority_queue< QueueElement, std::greater> pq;

    result.costs[startNode] = 0.0;
    pq.push({0.0, startNode});

    while (!pq.empty()) {
        auto topPair = pq.top();
        double currentCost = topPair.first;
        int u = topPair.second;
        pq.pop();

        if (currentCost > result.costs[u]) continue;

        for (const auto& edge : adjList[u]) {
            int v = edge.destination;
            double edgeWeight = optimizeForTime ? edge.getTravelTimeMinutes() : edge.distance;
            double newCost = result.costs[u] + edgeWeight;

            if (newCost < result.costs[v]) {
                result.costs[v] = newCost;
                result.distances[v] = result.distances[u] + edge.distance;
                result.travelTimes[v] = result.travelTimes[u] + edge.getTravelTimeMinutes();
                result.predecessors[v] = u;
                pq.push({newCost, v});
            }
        }
    }
    return result;
}
```

4.2 NavigationSystem – Turn-by-Turn Route Guidance Code

File: *src/NavigationSystem.cpp*

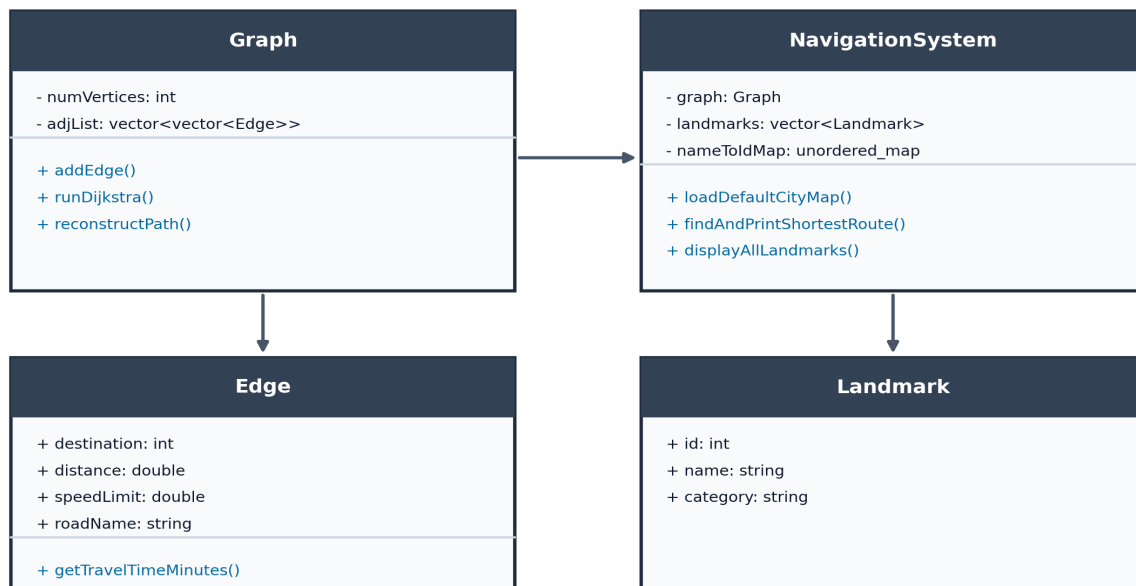
```
void NavigationSystem::findAndPrintShortestRoute(const std::string& srcName,
                                                const std::string& destName,
                                                bool optimizeForTime) const {
    int srcId = getLandmarkId(srcName);
    int destId = getLandmarkId(destName);

    DijkstraResult result = graph.runDijkstra(srcId, optimizeForTime);
    std::vector path = graph.reconstructPath(srcId, destId, result.predecessors);

    std::cout << "\n===== \n";
    std::cout << "                      OPTIMAL NAVIGATION ROUTE FOUND          \n";
    std::cout << "===== \n";
    for (size_t i = 0; i < path.size(); ++i) {
        std::cout << " Step " << (i + 1) << ": [" << path[i] << "]" << getLandmarkName(path[i]);
        if (i < path.size() - 1) {
            std::cout << "\n          |--> Drive along road segment\n";
        }
    }
    std::cout << " Total Distance   : " << result.distances[destId] << " km\n";
}
```

```
std::cout << " Est. Travel Time : " << result.travelTimes[destId] << " mins\n";  
}
```

4.3 Model Layer – Key Class Structures



4.4 System Snapshots & Execution Output

```

NaviPath Terminal - C++ Dijkstra Engine

=====
OPTIMAL NAVIGATION ROUTE FOUND
=====
Origin       : Central Station (ID: 0)
Destination  : Metropolitan Airport (ID: 5)
Optimization : Fastest Travel Time
=====
TURN-BY-TURN NAVIGATION INSTRUCTIONS:
=====
Step 1: [0] Central Station
         |--> Drive along 'Main Street' (3.5 km @ 40.0 km/h ~ 5.2 mins)
Step 2: [1] Downtown Hub
         |--> Drive along 'Tech Boulevard' (8.2 km @ 70.0 km/h ~ 7.0 mins)
Step 3: [3] Tech Park
         |--> Drive along 'Airport Highway' (12.0 km @ 90.0 km/h ~ 8.0 mins)
Step 4: [5] Metropolitan Airport (ARRIVED!)
=====
SUMMARY:
Total Distance : 23.70 km
Est. Travel Time : 20.3 mins
Number of Segments : 3 road segments
=====
  
```

CHAPTER 5: BIBLIOGRAPHY & PROJECT LINKS

- ISO C++17 Standard (ISO/IEC 14882:2017). <https://isocpp.org/std/the-standard>
- Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs". Numerische Mathematik. 1: 269–271.
- CMake Documentation & Build Automation. <https://cmake.org/documentation/>
- CipherSchools – Data Structures and Algorithms Training. <https://cipherschools.com>
- LeetCode – Algorithmic Practice Platform. <https://leetcode.com>

Project Links

The complete source code, documentation, build scripts, and tests for this project are publicly available at the link below:

GitHub Repository:

https://github.com/Shauryasharma0003/DSA_SHORTEST_PATH_NAVIGATION_SYSTEM_SHAURYA_SHARMA.git